

Proof catalog: every theorem, how it's proved, and the lemmas it leans on

Hoang Nguyen

Abstract

Every named theorem proved in `Gc/Model/` and `Gc/Reachability/Reachable/`, listed with a one-line statement, a rough sketch of the proof, and the key supporting lemmas it actually calls. `Gc/Reachability/Referencable/` is excluded (deprecated, superseded by `Reachable/`; see `CLAUDE.md`). Definitions and formal statements already given in full in `writeup/main.tex` are cited by name, not restated.

Contents

1	Notation	2
2	<i>Gc/Model: ValidConfig</i> preservation	2
2.1	<code>RuntimeConfig.start_valid</code>	2
2.2	<code>enter_valid</code> (<code>enter xf bridgeVar</code>)	2
2.3	<code>exit_valid</code> (<code>exit</code>)	3
2.4	<code>fieldAsgn_valid</code> (<code>fieldAsgn xf y</code>)	3
2.5	<code>makeObjRegion_valid</code> (<code>makeObjRegion x</code>)	3
2.6	<code>makeObjStack_valid</code> (<code>makeObjStack x</code>)	4
2.7	<code>makeRegion_valid</code> (<code>makeRegion x</code>)	4
2.8	<code>merge_valid</code> (<code>merge x</code>)	4
2.9	<code>varAsgn_valid</code> (<code>varAsgn x yf</code>)	5
2.10	<code>swap_valid</code> (<code>swap x yf</code>)	5
2.11	<code>allPreserve_ValidConfig</code>	6
3	<i>Gc/Reachability/Reachable: semantics and the confinement toolkit</i>	6
3.1	Semantics-level theorems (<code>Semantics.lean</code>)	6
3.2	The confinement toolkit (<code>Lemmas/Common.lean</code>)	6
3.3	<i>CR6</i> (<code>CR6.lean</code>)	7
4	<i>FrameReachableAtLaterFrame</i>	7
4.1	<code>RuntimeConfig.start_frameReachableAtLaterFrame</code>	7
4.2	The nine per-operation proofs	7
4.3	<code>frameReachableAtLaterFrame_step_all</code>	9
5	<i>StackReachable_invariant_for_suspended_region_objects</i>	9
5.1	<code>stackReachable_invariant_all</code>	10
6	What's not proved yet	10

1 Notation

$\langle S, H \rangle$: stack of frames, heap of regions. $\text{Rld}(rid)/\text{Old}(oid)$: references. $a.\text{loc}(\langle S, H \rangle)$: where a reference resolves ($\text{Stk}(fid)$, $\text{Rgn}(rid)$, or undefined). $a.\text{objAt}(\langle S, H \rangle)$: the object an **Old** names. The ten invariants $L1, L2, H1, H2, H3, S1, S2, S3, HS1, HS2$ making up *ValidConfig* are defined in `main.tex`; one-line reminders:

Invariant	What it rules out
$L1$	the same object id allocated twice
$L2$	a frame executing inside a <i>Closed</i> region
$H1$	a region whose bridge object isn't in its own <i>objMap</i>
$H2$	two live $\text{Rld}(rid)$ references to the same region
$H3$	a region's contents pointing outside that region
$S1$	two stack frames sharing a region
$S2$	a frame referencing a stack object owned by a later frame
$S3$	a frame referencing a region owned only by a later frame
$HS1$	a dangling Old
$HS2$	a dangling Rld

2 Gc/Model: *ValidConfig* preservation

Each `<op>_valid` theorem has the shape $\text{ValidConfig}(cfg) \rightarrow op\ cfg = \text{some } cfg' \rightarrow \text{ValidConfig}(cfg')$, proved by destructing the operation via its `<op>_cases` lemma and discharging the ten invariants one at a time. Below, each invariant is marked either trivial (with the one-line reason) or given its key lemma.

2.1 `RuntimeConfig.start_valid`

Base case: one root region (Open, bridge 0), one root frame. All ten invariants reduce to facts about a one- or two-element list, closed by `decide/simp`.

2.2 `enter_valid (enter xf bridgeVar)`

Pushes a new, empty frame into region rid' (found via `resolveFA xf`, must be Closed), flips rid' to Open.

L1 trivial: no *objMap* touched (new frame is empty; heap change is a status flip).

L2 the new frame's region is Open by construction; other frames' lookups survive the heap update at a different key.

H1 trivial: the status flip doesn't touch *bridgeObjectId/objMap*.

H2 trivial: no reference created, removed, or relocated.

H3 `enter_corollary_2`: $\text{loc}(\cdot)$ is unconditionally unaffected (only a status scalar changes), so existing $H3$ witnesses transfer directly.

S1 `enter_corollary_1` (region-uniqueness from $L1$) plus $L2$'s contrapositive: a Closed region (what `enter` requires) can't already be owned by an on-stack frame, so the new frame's region is distinct from every existing one.

S2, S3 the new frame is empty (nothing to check for it); existing frames rely on `enter_corollary_2`.

HS1, HS2 trivial: nothing new enters *refs*.

2.3 exit_valid (exit)

Pops the last frame, flips its region back to Closed.

L1 trivial: the popped frame’s ids are simply dropped, and by *L1* on *cfg* they were already disjoint from everything remaining.

L2 surviving frames’ regions are untouched by the status flip except needing “region $\neq$ the popped one,” from *S1*.

H1 trivial: status flip only.

H2 trivial: no reference touched.

H3 the popped frame’s contents vanish with it; survivors’ locations are unaffected.

S1, S2, S3 index bookkeeping over the shrunk stack (survivors keep their indices; nothing points at the now-gone last slot). *Naming note:* in **Preservation/Exit.lean** these are proved by *exit_S2*, *exit_S3*, *exit_S4* respectively (an internal off-by-one in the file’s own lemma names, not in the mathematics).

HS1, HS2 the popped frame’s refs disappear; nothing new appears.

2.4 fieldAsgn_valid (fieldAsgn xf y)

Overwrites one field of an object already resolved via **resolveV**, either on the stack (same frame) or inside the writer’s own Open region (target must resolve into that same region too).

L1, L2, S1, H1 trivial: no id allocated, no frame/region/bridge changed, only a field value inside an already-present object.

H2 the new value is always an Old (this rule never stores an Rld in a field), so overwriting can only decrease-or-preserve the Rld-count: *fieldAsgn_corollary_object_insert_count_le* lifted through *_objMap_insert_bind_refs_count_le/_heap_insert_refs_count_le*.

H3 the region branch’s own side condition (new value resolves into the *same* region as the target) is exactly what *H3* needs; formalized via *fieldAsgn_corollary_region_loc_eq/_region_heap_objectIds_perm*.

S2, S3 both *xf.root* and *y* were already resolvable (hence already index-bounded) before the write; *fieldAsgn_corollary_resolveV_loc_ancestor* carries that bound over to the newly-stored occurrence.

HS1 *fieldAsgn_corollary_resolveV_oid_mem_objectIds*: the written value was already live, so already counted in *objectIds*.

HS2 trivial: no Rld ever passes through this rule.

2.5 makeObjRegion_valid (makeObjRegion x)

Allocates a fresh, empty object directly inside the writer’s own Open region.

L1 *RuntimeConfig.freshObjectId_not_mem*: the fresh id can’t already be present.

L2, S1 trivial: no frame/region status or ownership changes.

H1 trivial: the pre-existing bridge id is untouched by inserting a new key.

H2 *makeObjRegion_corollary_heap_refs_perm*: the new object is empty (no refs), so *Heap.refs* is an exact permutation of before.

H3 the fresh object trivially satisfies it (it’s the region just written); other objects rely on *makeObjRegion_corollary_loc_eq* (needs *makeObjRegion_corollary_region_unique* to handle the heap-entry reorder from inserting at an already-present region key).

S2, S3 *makeObjRegion_corollary_loc_fresh*: the fresh object always resolves into the heap (never the stack), so *S2* is vacuous for it and *S3* is trivial (owning frame is the current one).

HS1 trivial: the fresh object is now in *objectIds* by construction.

HS2 trivial: no Rld touched.

2.6 makeObjStack_valid (makeObjStack x)

Allocates a fresh, empty object directly on the current frame’s own (stack-local) *objMap*; the heap is untouched entirely (**makeObjStack_corollary_1**).

L1 **freshObjectId_not_mem**, on the stack side.

L2, H1, H2 trivial: heap untouched.

H3 doesn’t apply to the fresh object (it resolves to *Stk*, not *Rgn*); heap side is untouched for everything else.

S1 trivial.

S2, S3 the fresh object resolves to the current (last) frame’s own index, trivially within bound; other objects are unaffected (insert at a fresh key).

HS1, HS2 trivial.

2.7 makeRegion_valid (makeRegion x)

Allocates a fresh Closed region with its own fresh bridge object, storing Rld to it in the current frame’s variable *x*.

L1 **freshObjectId_not_mem** for the new bridge object.

L2 trivial for existing frames (insert at a genuinely fresh heap key disturbs nothing); nobody executes in the new region yet.

H1 trivial by construction: the fresh region’s only object is its own bridge.

H2 the new Rld is stored exactly once (in *x*), and a not-yet-existing region id can’t already have been referenced (old config’s *HS2* plus **freshRegionId_not_mem**).

H3 trivial for the fresh region; **makeRegion_corollary_loc_eq** for everything else (fresh-key insert, no reorder needed).

S1 trivial: no existing frame’s regionId changes, and the new region isn’t any frame’s.

S2, S3 the fresh bridge object resolves into the heap (never the stack); its owning frame is the one that just created it.

HS1 trivial: fresh bridge object now in *objectIds*.

HS2 trivial: fresh region id now in *heap.keys* by construction.

2.8 merge_valid (merge x)

Folds a Closed region *rid*’ into the current Open region *rid*: combines their *objMaps*, deletes *rid*’, rebinds *x* to *rid*’s old bridge object. The busiest file (~1200 lines), the only operation deleting a heap key.

The load-bearing fact the two regions’ object-id sets are disjoint: **merge_corollary_rid_ne_regionId** (Open $\neq$ Closed) plus **merge_corollary_region_unique** (an *L1* corollary: no id in two heap entries). This alone rules out the union silently colliding or dropping an object, since **AList.union** in general erases duplicate keys from one side.

L1 **merge_corollary_union_append** (disjoint keys $\Rightarrow$ union is a plain append) plus **merge_corollary_heap_entries_perm** (the post-merge heap is a permutation of the merged pair plus everything untouched, via two nested **List.exists_of_kerane** extractions): *L1* then follows from permutation-invariance of **Nodup**.

L2, S1 trivial: only the last frame’s *varMap* and the two regions’ heap entries change.

H1 the merged region’s bridge object (*rid*’s own, unchanged) is still present, since the union only adds keys.

- H2** the deleted reference $\text{Rld}(rid')$ simply disappears from *refs*; the new *Old* written to *x* was already counted under *H1* of *rid'*.
- H3** `merge_corollary_loc_of_mem`: an object from either original region still resolves into the merged *rid*, since the stack side of `.loc(·)` is untouched (only *x*'s *varMap* entry changes) and the heap side's uniqueness (from *L1* above) pins *rid*'s merged entry as the only match.
- S2, S3** carried by the same `merge_corollary_loc_of_mem/index-split` machinery as *H3*; `merge_corollary_stackWithIndex_split` handles the shared “last frame changed, rest didn't” bookkeeping.
- HS1, HS2** follow from the permutation facts above: nothing created or destroyed, only relocated.

2.9 varAsgn_valid (varAsgn x yf)

Two branches: rebind the frame's own bridge variable to an already-in-region value (“bridge branch”), or bind a fresh local variable to a resolved value (“fresh-var branch”, requiring `resolveV x cfg = none`).

- L1** trivial both branches: no object created; bridge branch permutes *heap.objectIds* via a re-order (insert at an existing key), not a content change (`varAsgn_corollary_bridge_heap_objectIds_perm`).
- L2, S1** trivial: no region status or frame *regionId* touched.
- H1** bridge branch: the new bridge id is already inside the region by the rule's own side condition ($yfRef.loc(\cdot) = \text{Rgn}(rid)$, $rid = frame.regionId$). Fresh-var: trivial.
- H2** `varAsgn_corollary_bridge_heap_refs_perm`: *bridgeObjectId* isn't read by *Region.refs* at all, so the bridge branch's heap refs are literally unchanged; fresh-var branch only relocates an existing reference into a new var slot.
- H3** `varAsgn_corollary_loc_eq`: `.loc(·)` is unconditionally preserved in both branches (literal equality for fresh-var; uniqueness-based, `varAsgn_corollary_region_unique`, for the bridge branch's heap reorder).
- S2, S3** fresh-var branch: `varAsgn_corollary_resolveV_loc_ancestor / _yfRef_ancestor` show the copied value already had a valid owning ancestor frame on the stack before the copy.
- HS1** fresh-var branch: `varAsgn_corollary_yfRef_mem_refs` (the copied value was already live).
- HS2** trivial: no *Rld* is ever created or moved by `varAsgn`.

2.10 swap_valid (swap x yf)

The busiest rule syntactically (four branches: stack, region-object, region-region, region-bridge), each exchanging two live reference slots at once. Largest file (~2250 lines).

- L1, L2, S1** trivial: no allocation, only reassigning values at already-present keys.
- H1** the region-bridge branch rebinds *bridgeObjectId*; same argument as `varAsgn`'s bridge branch (new bridge already inside the region, by the rule's side condition).
- H2** `swap_corollary_alist_insert_count_eq / _objMap_insert_bind_count_eq / _heap_insert_bind_count_eq`: since `swap` is a genuine exchange (the two count-deltas from displacing one value and inserting another cancel exactly), the overall *Rld*-count is provably unchanged, not merely bounded.
- H3** the region-touching branches' own side conditions keep everything inside one Open region; `swap_corollary_region_loc_eq / _stack_loc_eq` carry pre-existing objects' locations across.
- S2, S3** (the two largest proofs in the file) need both the displaced old value and the newly-copied value to respect index bounds simultaneously; `swap_corollary_stack_field_eq_yfRef / _region_field_eq_yfRef` confirm the overwritten field held exactly the value being displaced

(a genuine exchange, not a blind overwrite), and existing $S2/S3$ facts about both slots' pre-mutation values carry over.

HS1, HS2 both displaced values were already live before the swap, so nothing new enters *refs*.

2.11 allPreserve_ValidConfig

Case-splits on the reified *Stmt* type (`Mutation/Stmt.lean`) and dispatches to the nine theorems above. No independent content.

3 Gc/Reachability/Reachable: semantics and the confinement toolkit

3.1 Semantics-level theorems (`Semantics.lean`)

ReachableStep, *RegionReachable*, *FrameReachable*, *StackReachable*, *FrameRoot* are defined in full in `main.tex`.

RegionReachable_iff_reflTransGen $RegionReachable(rid, ref) \iff \exists region, H(rid)=region \wedge ReflTransGen(\rightarrow)(Old(region.bridgeObjectId)) ref$. Proof: the inductive **bridge/step** constructors correspond exactly to *ReflTransGen*'s **refl/tail**; induction on either side transfers directly.

FrameReachable_iff_reflTransGen Same idea with two possible roots (*FrameRoot*: a variable's value, or the frame's own bridge object) instead of one.

RegionReachable_implies_FrameReachable If frame *F* owns region *rid*, $RegionReachable(rid, \cdot) \rightarrow FrameReachable(F.index, \cdot)$: induction on the *RegionReachable* witness, replacing the **bridge** base case with *FrameReachable.bridge*.

3.2 The confinement toolkit (`Lemmas/Common.lean`)

stack_step_index_le $S2$ restated at the *ReachableStep* level: a $stack \rightarrow stack$ hop's target frame index is $\leq$ the source's. Proof: unfold *objAt?* at the source, apply $S2$ directly to the resulting membership fact.

successor_of_region_object $H3$ forward, at the *ReachableStep* level: stepping out of an object in region *rid* lands back in *rid*. Proof: unfold *objAt?*, apply $H3$.

predecessor_of_region_object Backward confinement, Open region: anything stepping *into* an in-region object is itself in that region or on the stack, never a bare *Rld* or a different region. Proof: case on the predecessor; an *Rld* source is ruled out by **open_rid_no_step** (same region) or by $H3$ forcing the other region's bridge object's own location to disagree; an *Old* source is reclassified by $H3$ applied to whatever it currently resolves into.

predecessor_of_closed_region_object Same statement, Closed region: the "on stack" branch is now impossible (**closed_region_not_owned**, $L2$'s contrapositive), and the *Rld* branch (the portal) becomes an admissible predecessor instead of a contradiction.

predecessor_of_stack_object Backward confinement, stack: anything stepping into a stack-resident object is itself stack-resident (an *Rld* or region-object predecessor is ruled out via $H3$ forcing a *Rgn* location, contradicting *Stk*).

SafeRef / predecessor_safe / safe_reflTransGen_transport $SafeRef_{rid}(ref)$: in *rid* or on the stack, never a bare *Rld*. Safety propagates backward one hop (combines the two predecessor lemmas above); if *ReachableStep* agrees on every *Old*-sourced step between cfg, cfg' , any chain ending at a safe target transports wholesale (induction on the chain, applying **predecessor_safe** at each step).

stack_container_confined Anything *FrameReachable*-reaching a stack-resident object at index fid_C is rooted from index $\geq fid_C$. *Proof*: restate as **ReflTransGen**, induct forward from the fixed root generalized over the chain’s (growing) end, applying **predecessor_of_stack_object** and **stack_step_index_le** at each hop; the two root shapes close via *S2* directly.

stack_step_region_index_le *S3* restated at the **ReachableStep** level (the region-side analogue of **stack_step_index_le**).

region_container_confined Anything reaching an object inside an *Open* region owned by frame *ownerFrame* is rooted from index $\geq ownerFrame.index$. *Proof*: same shape as **stack_container_confined**, dispatching on **predecessor_of_region_object**’s two surviving cases (stay in-region, or escape to stack via **stack_step_region_index_le** plus **merge_corollary_regionId_unique_index** to convert “same region” into “same index”).

FrameReachable_extend Extends a *FrameReachable* witness forward along any further **ReflTransGen** chain (one line: **FrameReachable.step** repeatedly).

RegionReachable_of_FrameReachable_closed For a *Closed* region: any *FrameReachable* witness to an in-region object is already a *RegionReachable* one. *Proof*: backward induction over the chain, dispatching on **predecessor_of_closed_region_object** (continue in-region, or stop at the portal hop).

FrameReachable_rid_of_closed_container Companion: reaching anything inside a *Closed* region forces the region’s own **Rld** to already be *FrameReachable* too. Same backward induction, different conclusion at the portal-hop case.

3.3 CR6 (CR6.lean)

StackReachable_iff_RegionReachable_of_closed For $H(rid)$ *Closed* and *oid* inside it: *given* $Rld(rid)$ is already *StackReachable*, $StackReachable(Old(oid)) \iff RegionReachable(rid, Old(oid))$. *Proof*: forward direction is **RegionReachable_of_FrameReachable_closed** directly (doesn’t even need the hypothesis); backward direction glues the portal hop onto the already-reachable $Rld(rid)$ via **FrameReachable_extend**.

not_StackReachable_rid_implies_not_StackReachable_objects_of_closed If $Rld(rid)$ is *not* stack-reachable, nothing inside the *Closed* region is either. *Proof*: contrapositive of **FrameReachable_rid_of_closed_container**.

4 FrameReachableAtLaterFrame

Statement (**main.tex**’s *FRALF*): an object in an earlier frame’s region, already frame-reachable from a later frame, is already frame-reachable from its own frame. **FrameReachableAtLaterFrame_step** **cmd** is its single-step-preservation form.

4.1 RuntimeConfig.start_frameReachableAtLaterFrame

Vacuous: a single frame makes $F.fid < F'.fid$ unsatisfiable.

4.2 The nine per-operation proofs

Five distinct shapes appear.

(A) **Additive, via whole-chain safety transport — enter, exit.** Nothing is allocated and no existing content changes (only a status flip). Every chain ending at the tracked object is shown

SafeRef and transported wholesale.

`frameReachableAtLaterFrame_step_enter` Key lemmas: `enter_oid_step_iff` (step relation agrees on Old-sourced steps), `safe_reflTransGen_transport`, `enter_regionId_ne`, `enter_frame_mem_up/down`. The one wrinkle: if the “later frame” turns out to be the freshly-pushed one, its only possible root is the entered region’s bridge object, and a safety argument shows that can’t be the (different) tracked object.

`frameReachableAtLaterFrame_step_exit` Key lemmas: `exit_frame_reachable_transport/_backward` (built from `exit_avoid_popped_step`, `exit_oid_step_iff_of_ne_popped`, and `safe_reflTransGen_root_safe`), `exit_regionId_ne`.

(B) Additive, fresh entity is a dead end — `makeObjStack`, `makeObjRegion`, `makeRegion`. Something new is allocated, but any chain trying to route through it collapses to a contradiction (a fresh id has no stored refs and doesn’t yet resolve anywhere).

`frameReachableAtLaterFrame_step_makeObjStack` Key lemmas: `makeObjStack_frame_reachable_-iff` (pre-existing frames below the last index), `freshObjectId_loc_ne_rgn`, `freshObjectId_objAt_none` (the fresh id’s chain is a dead end: it doesn’t resolve into the tracked region, and it has no outgoing step either).

`frameReachableAtLaterFrame_step_makeObjRegion` Same shape; key lemmas `makeObjRegion_frame_reachable_iff`, `makeObjRegion_regionId_ne`, plus the same two fresh-id dead-end facts.

`frameReachableAtLaterFrame_step_makeRegion` Same shape, one level richer (a fresh region *and* its fresh bridge object): `makeRegion_frame_reachable_iff`, `makeRegion_reflTransGen_transport_down` (chains avoiding the fresh Rld transport freely), `makeRegion_corollary_loc_fresh`, `freshRegionId_no_step`. A chain hitting the fresh Rld can take at most one more hop (into the equally-fresh, empty bridge object) before dying.

(C) Root-substitution — `varAsgn`. `varAsgn_step_eq` shows the *entire* `ReachableStep` relation is literally unchanged (even the bridge-var branch: the touched region stays Open, so no Rld-step existed either side to disturb). Only a variable’s *bound value* changes, to something already independently reachable.

`frameReachableAtLaterFrame_step_varAsgn` Key lemmas: `varAsgn_step_eq`, `varAsgn_frame_reachable_iff` (pre-existing frames), and, for the mutated frame’s own new root, `resolveFA_frameReach` (the field value being copied already had an independent `FrameReachable` witness) combined with `region_container_confined` to bound where that witness’s frame can sit relative to the tracked frame.

(D) Content-mutating, genuine escape induction — `fieldAsgn`, `swap`. The step relation changes at exactly the mutated container. See the detailed technique note below.

`frameReachableAtLaterFrame_step_fieldAsgn` Key lemmas: `fieldAsgn_step_iff_of_ne` (step relation agrees off the mutated id `oidC`), `fieldAsgn_frame_root_iff`, `region_container_confined/stack_container_confined` (bounding `oidC`’s own owner index), then a `Relation.ReflTransGen` induction on walk checking each hop against `fieldAsgn_stack_oidC_step_of_ne_oidY/_region_oidC_step_of_ne_oidY`; a match escapes onto the field’s new value via `resolveV_frameRoot`, and `reflTransGen_region_open_ne_absurd` rules out the region-branch’s new value ever reaching a *different* Open region.

`frameReachableAtLaterFrame_step_swap` Same shape across all four branches; key lemmas `swap_frame_reachable_iff_of_lt` (suspended chains can’t reach the mutated container at all, by

the owner-index bound, so no escape machinery is even needed for them), `swap_escape/swap_root_escape/swap_root_escape_rid` (the shared “new value has its own independent root” argument, reused across branches), `swap_stack_void_step_of_ne/_region_stack/_bridge_` (per-branch step-agreement off the mutated id), `reflTransGen_rid_source_open_absurd`.

(E) Reference-deleting — merge. See the walkthrough note below; the key fact is that a live, about-to-be-deleted `Rld(rid')` can never be a step *target*.

`frameReachableAtLaterFrame_step_merge` Key lemmas: `merge_ridPrime_not_step_target` (*H2*-based: a predecessor would push the count past 1), `merge_chain_transport_down` (any `Rld(rid')`-avoiding chain transports for free), `merge_ridPrime_reflTransGen_iff` (a chain rooted at `Rld(rid')` is the same chain, one hop shorter, rooted at its bridge object instead), `merge_frame_reachable_iff`, `merge_regionId_ne_ridPrime`.

Technique note (D). The step relation changes at exactly one container, id `oidC`. `region_container_confined/stack_container_confined` bound `oidC`’s reachable set to frame indices $\geq$ the active (mutating) frame’s own index, `fidactive`. Any chain confined strictly below `fidactive` never touches `oidC` and transports untouched — this covers every strictly-suspended frame automatically. The one remaining case is a chain rooted exactly at the active frame, walked forward hop by hop (`Relation.ReflTransGen.head_induction_on`): at the one hop that could be the newly-written edge, the chain escapes onto the field’s new value, which has its own independent root (`resolveV_frameRoot`), reducing the argument back to the already-handled confined case.

4.3 frameReachableAtLaterFrame_step_all

Dispatcher, case-splitting on *Stmt*.

5 StackReachable_invariant_for_suspended_region_objects

Statement (`main.tex`): for a suspended frame *F* and an object *oid* in *F*’s region, `StackReachable(Old(oid))` is unchanged by any single operation, *given FRALF* holds beforehand. Proved as an $\iff$, not a one-way implication.

Each `StackReachableInvariant/⟨Op⟩.lean` reuses *exactly* the case analysis of its §3-numbered *FRALF* sibling above (same key lemmas per operation), applied in both directions: “reachable before $\Rightarrow$ reachable after” is close to a direct instance of the *FRALF*-style transport; “reachable after $\Rightarrow$ reachable before” is where the escape argument (shapes D/E) is actually exercised, since the newly-written edge or the about-to-be-deleted reference exists on the `cfg'  $\rightarrow$  cfg` side, not the other.

`stackReachable_invariant_enter` Both directions via `safe_reflTransGen_transport` (called with the step-iff in each direction).

`stackReachable_invariant_exit` Both directions via `exit_reflTransGen_transport/_backward`.

`stackReachable_invariant_makeObjStack / makeObjRegion / makeRegion` Both directions via the corresponding `⟨op⟩_frame_reachable_iff`, with the fresh-id dead-end lemmas closing the one new case per direction.

`stackReachable_invariant_varAsgn` Both directions via `varAsgn_step_eq + varAsgn_frame_reachable_iff`; the root-substitution argument (`resolveFA_frameReach`) is needed in the forward direction here (a suspended-frame witness reaching the pre-mutation root must still resolve after the var is rebound).

`stackReachable_invariant_fieldAsgn` Forward direction: confined transport suffices (the mutated id's owner-index bound already excludes it from any suspended-frame chain). Backward direction: the full escape argument (shape D), since the new edge exists only in *cfg'*.

`stackReachable_invariant_swap` Same asymmetry as `fieldAsgn`, across all four branches.

`stackReachable_invariant_merge` Forward: confined transport via `merge_chain_transport_down`. Backward: the `merge_ridPrime_reflTransGen_iff` substitution, since the deleted reference's replacement only exists in *cfg'*.

5.1 `stackReachable_invariant_all`

Dispatcher, case-splitting on *Stmt*.

6 What's not proved yet

No **Guarantees.lean**-style top-level GC-safety theorem sits on top of *StackReachable_invariant_for_suspended_region_objects*. Every theorem in §4–5 is a single-step fact; composing it over an arbitrary-length trace (the literal report.pdf claim, whose qualifier is invariance for as long as a region *stays* suspended, not just across one operation) is not currently being pursued.